

Text Spam Detector

Meera Aleem
University of Illinois
Urbana-Champaign
Champaign, Illinois, USA
meeraaa2@illinois.edu

Elijah John
University of Illinois
Urbana-Champaign
Champaign, Illinois, USA
elijah7@illinois.edu

Rabia Khan
University of Illinois
Urbana-Champaign
Champaign, Illinois, USA
rkhan84@illinois.edu

Isra Mohammad
University of Illinois
Urbana-Champaign
Champaign, Illinois, USA
imoha28@illinois.edu

Hannah McGowan
University of Illinois
Urbana-Champaign
Champaign, Illinois, USA
hannahm9@illinois.edu

Abstract

In this project, we are developing an automated SMS spam detection system that classifies text messages as either spam or legitimate (ham) using machine learning and natural language processing techniques. The system is designed to help identify unwanted or potentially harmful messages while ensuring legitimate texts are not incorrectly flagged. Our pipeline includes preprocessing raw SMS text, extracting meaningful features using TF-IDF vectorization, and applying supervised learning models such as Naive Bayes and Logistic Regression. The goal is to achieve high accuracy while maintaining strong precision and recall so the model is reliable in practice. So far, we have finalized our dataset, the UCI SMS Spam Collection dataset, implemented initial preprocessing steps, and begun exploratory data analysis. We have also started building a draft version of our pipeline and preparing for early model evaluation.

Keywords

spam detection, NLP, machine learning, TF-IDF, text classification, SMS spam detection

1 Summary

In this project, we are developing an automated SMS spam detection system that classifies text messages as either spam or legitimate (ham) using machine learning and natural language processing techniques. The system is designed to help identify unwanted or potentially harmful messages while ensuring legitimate texts are not incorrectly flagged.

Our pipeline includes preprocessing raw SMS text, extracting meaningful features using TF-IDF vectorization, and applying supervised learning models such as Naive Bayes and Logistic Regression. The goal is to achieve high accuracy while maintaining strong precision and recall so the model is reliable in practice.

So far, we have finalized our dataset, the UCI SMS Spam Collection dataset, implemented initial preprocessing steps, and begun exploratory data analysis. We have also started building a draft version of our pipeline and preparing for early model evaluation.

2 Introduction

Spam text messages remain a common issue and are widely used for scams, phishing attempts, and unwanted advertising. Because SMS messages are short and often written informally, detecting spam is a challenging text classification problem. Building an SMS spam detector provides useful insight into how machine learning and natural language processing can be applied to real-world messaging data.

Spam detection requires identifying patterns in unstructured text while balancing false positives and false negatives. A false positive occurs when a legitimate message is classified as spam, while a false negative occurs when a spam message goes undetected. Since spam language constantly changes, a strong detector must generalize well across different writing styles and message patterns.

Our project addresses this problem by building a multi-stage pipeline that processes raw SMS data, extracts relevant textual features, and applies machine learning models to classify each message as spam or ham. This milestone focuses on the progress we have made so far, including dataset validation, preprocessing, and our early evaluation plan.

3 Data Description and Preprocessing

We are using the UCI SMS Spam Collection dataset, a publicly available labeled dataset containing SMS messages classified as either spam or ham. This dataset is widely used for spam detection research and is appropriate for our project because it provides a clear binary classification task with enough labeled examples for training and evaluation.

3.1 Preprocessing Steps

To prepare the data for modeling, we implemented several preprocessing techniques:

- Converted all text to lowercase to ensure consistency
- Removed punctuation, numbers, and special characters
- Tokenized text into individual words
- Removed stopwords (common words like “the” and “and”) to reduce noise
- Applied basic text cleaning to standardize input

These steps help reduce noise in the dataset and improve the quality of the features extracted later in the pipeline.

3.2 Exploratory Observations (Initial)

From our initial exploration of the SMS Spam Collection dataset, we observed that:

- Spam messages tend to use more promotional, urgent, or persuasive language
- Ham messages are generally more casual, personal, and varied in wording
- Spam messages often contain repetitive keywords related to prizes, offers, or calls to action

This preprocessing ensures that our dataset is clean, structured, and suitable for feature extraction and classification.

4 Initial Evaluation

We implemented a basic version of our pipeline using TF-IDF vectorization and prepared to train an initial classification model such as Naive Bayes.

4.1 Pipeline (Draft)

Raw SMS Text → Cleaning → Tokenization → Stopword Removal → TF-IDF → Model → Prediction

4.2 Preliminary Results (Expected Structure)

At this milestone stage, our evaluation pipeline is being set up, and we plan to report the following metrics:

- Accuracy
- Precision
- Recall
- F1 Score

We also plan to generate a confusion matrix to better understand model behavior:

- True Positives: correctly identified spam messages
- True Negatives: correctly identified ham messages
- False Positives: ham messages incorrectly labeled as spam

- False Negatives: spam messages incorrectly labeled as ham

Although full evaluation is still in progress, the pipeline structure is in place and gives us a strong starting point for measuring model performance.

5 Insights So Far

From our work so far, several key insights have emerged:

- Data quality is critical: preprocessing strongly affects how useful the extracted features will be
- Spam detection is nuanced: small changes in wording can affect how a message is classified
- Precision is especially important: misclassifying legitimate text messages as spam would reduce trust in the system
- Model choice affects tradeoffs: different classifiers may perform differently on precision versus recall

Additionally, we now have a more realistic understanding of the project scope. While strong performance is achievable on the UCI SMS Spam Collection dataset, balancing all evaluation metrics and ensuring a reliable classifier requires careful experimentation and tuning.

Moving forward, we plan to:

- Improve feature extraction techniques
- Compare multiple models such as Naive Bayes, Logistic Regression, and SVM
- Optimize hyperparameters
- Expand evaluation with deeper analysis of classification errors

6 Exploratory Data Analysis

6.1 Dataset Overview

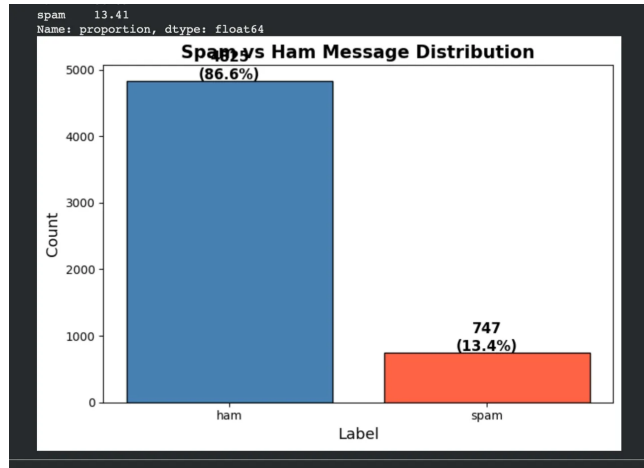
The SMS Spam Collection dataset, sourced from the UCI Machine Learning Repository, contains a total of 5,572 labeled SMS messages. Each message is classified as either spam or ham (legitimate). As shown in Figure 1, the dataset is imbalanced: 4,825 messages (86.6%) are ham and 747 messages (13.4%) are spam. This reflects realistic conditions, as spam messages represent a minority of real-world SMS traffic. Despite the imbalance, the dataset provides sufficient spam examples for training a classification model, and the imbalance itself is an important consideration when evaluating model performance, particularly for precision and recall metrics.

6.2 Message Length Analysis

One of the most informative structural features of the dataset is message length. As shown in Figure 2 and summarized in Table 2, spam and ham messages differ significantly in their character counts. Spam messages have an average length of

Table 1: Dataset Label Distribution

Label	Count	Percentage
Ham	4,825	86.6%
Spam	747	13.4%
Total	5,572	100%

**Figure 1: Spam vs. ham message distribution in the SMS Spam Collection dataset.**

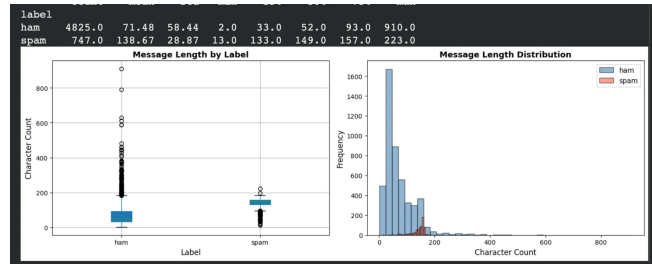
138.67 characters, nearly double the ham average of 71.48 characters. The median spam message is 149 characters, while the median ham message is only 52 characters. This suggests that spam messages tend to be longer, likely because they pack in promotional language, instructions, and contact information. Ham messages, by contrast, are shorter and more varied, ranging from 2 to 910 characters, reflecting the casual and conversational nature of legitimate texts.

Table 2: Message Length Statistics by Label

Label	Count	Mean Length	Median	Min	Max
Ham	4,825	71.48	52.0	2	910
Spam	747	138.67	149.0	13	223

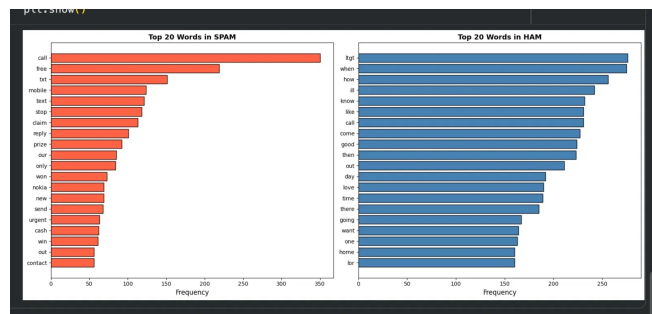
6.3 Word Frequency Analysis

Figure 3 displays the top 20 most frequent words in spam and ham messages after removing common stopwords. The contrast between the two categories is striking. Spam messages are dominated by action-oriented and promotional terms—the most frequent being “call” (350 occurrences), followed by “free” (220), “txt” (150), “mobile,” “text,” “stop,” “claim,” “reply,” “prize,” and “urgent.” These words reflect the persuasive and

**Figure 2: Message length analysis by label, including distribution and summary visualization.**

commercial nature of spam, often urging the recipient to take immediate action or claim a reward.

Ham messages, on the other hand, feature conversational and relational language such as “when,” “how,” “know,” “like,” “come,” “good,” “love,” and “time.” The presence of “ltgt” in ham is an artifact of HTML encoding in some messages and would be addressed during preprocessing. The clear lexical separation between spam and ham vocabulary confirms that word-level features will be highly informative for classification.

**Figure 3: Top 20 most frequent words in spam and ham messages after stopword removal.**

6.4 Vocabulary Size Analysis

Figure 4 reveals another meaningful difference between the two categories. Ham messages contain 7,402 unique words, while spam messages contain only 2,260 unique words—despite spam having been filtered from a much smaller sample. This means spam relies on a narrower, more repetitive vocabulary, while ham messages reflect a much broader and more diverse range of language. This finding supports the use of TF-IDF vectorization as a feature extraction method, since it will reward terms that are distinctive to one class and penalize terms that appear across all messages.

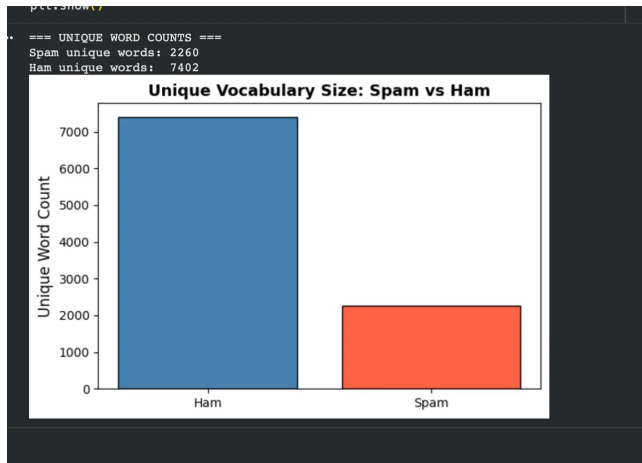


Figure 4: Unique vocabulary size comparison between ham and spam messages.

6.5 Dataset Adequacy Justification

The SMS Spam Collection dataset is well-suited for this spam detection project for several reasons. First, it is publicly available, fully labeled, and contains no missing values, making it immediately usable without extensive data repair. Second, at 5,572 messages, it provides enough volume to support an 80/20 train-test split while still yielding meaningful evaluation results. Third, the linguistic patterns identified through EDA—differences in message length, vocabulary size, and word choice—confirm that the dataset contains the kinds of distinguishing signals that machine learning classifiers can learn from. Finally, the dataset has been widely used in published spam detection research, lending credibility to its use as a benchmark.

7 System Architecture

7.1 System Overview

The text message spam detection system is designed as a multi-stage supervised classification pipeline. At a high level, the system ingests raw SMS messages from a labeled dataset, subjects them to a series of text normalization and preprocessing steps, transforms the cleaned text into a numerical feature representation, trains and compares multiple machine learning classifiers, and ultimately produces a binary spam-or-ham label for each message along with a diagnostic report of observed spam patterns.

SMS messages present a distinct set of challenges compared to other text classification domains. Messages are extremely short, typically 10 to 30 tokens after preprocessing, which limits the amount of discriminative signal available per sample and makes traditional bag-of-words representations sparser than in document-level tasks. Additionally, SMS

language is informal and abbreviation-heavy, with unconventional spellings, numeric shorthand (e.g., “2”, “u”, “ur”), and emoticons that do not appear in standard corpora. The pipeline architecture accounts for these properties at each stage.

The architecture is intentionally modular: each stage is implemented as an independent, composable component so that individual modules—the vectorizer, the classifier, the evaluator—can be swapped or tuned without affecting the rest of the pipeline. The system is implemented in Python using a Jupyter notebook, with scikit-learn as the core machine learning library and NLTK for linguistic preprocessing.

7.2 Pipeline Diagram

Figure 5 illustrates the end-to-end pipeline. The system proceeds in a linear, single-pass fashion from raw SMS data through preprocessing, feature extraction, model training, and evaluation to produce a final classified output.

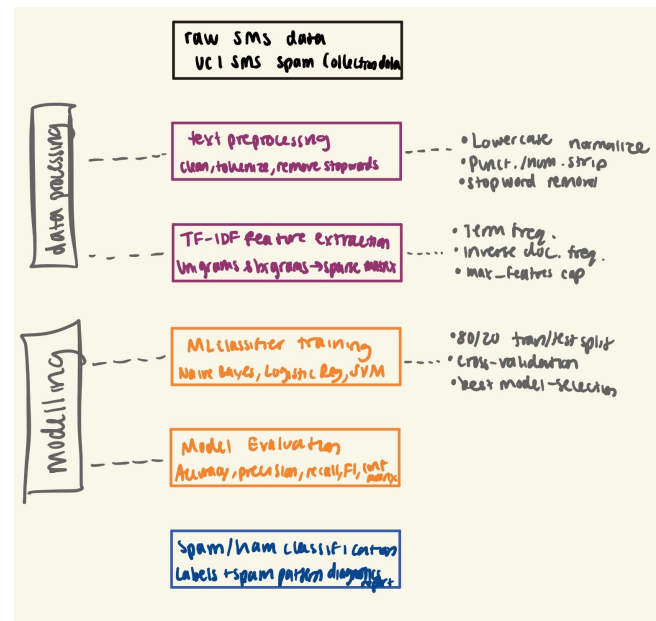


Figure 5: End-to-end pipeline of the text message spam detection system. Each stage is a discrete processing module.

Table 3 below enumerates each pipeline stage, the corresponding system component, and the specific methods or tools employed.

7.3 Stage-by-Stage Description

Stage 1 — Raw SMS Data. The pipeline is built on the UCI SMS Spam Collection dataset, a widely used benchmark corpus for SMS-based spam classification. The dataset contains 5,572 messages in English, of which 747 are labeled

Table 3: Pipeline Stages, Components, and Associated Methods for the SMS Spam Detection System

Pipeline Stage	Component	Description / Methods
Stage 1	Raw SMS Data	UCI SMS Spam Collection dataset (5,572 labeled messages: 747 spam, 4,825 ham)
Stage 2	Text Preprocessing	Lowercase normalization, punctuation/number stripping, tokenization, stopword removal (NLTK), lemmatization (WordNetLemmatizer)
Stage 3	TF-IDF Feature Extraction	Unigram and bigram vectorization (TfidfVectorizer); sublinear TF scaling; max_features cap to bound vocabulary
Stage 4	ML Classifier Training	Multinomial Naive Bayes, Logistic Regression (L2), Linear SVM; 80/20 stratified train-test split; 5-fold cross-validation
Stage 5	Model Evaluation	Per-class accuracy, precision, recall, F1-score; confusion matrices; best model selected by F1 with tie-break on precision
Output	Spam / Ham Prediction	Binary label (spam / ham) with confidence scores; spam pattern diagnostic report (top discriminative terms, pattern categories)

spam and 4,825 are labeled ham (legitimate). The class imbalance (approximately 13% spam) reflects realistic distributions observed in mobile messaging traffic and is an important consideration during training and evaluation. Each record in the dataset consists of a label (“spam” or “ham”) and the raw message text; no additional metadata is provided, which is consistent with the constraints of real-world SMS classification where only message content is available.

Stage 2 — Text Preprocessing. Raw SMS text is passed through a preprocessing sub-pipeline tailored to the properties of informal, short-form messaging. The steps are applied in the following order: (i) lowercasing to reduce surface-form variation; (ii) removal of punctuation and numeric characters, which carry limited discriminative signal relative to their noise contribution in short texts; (iii) word tokenization using NLTK’s `word_tokenize`; (iv) stopword removal using NLTK’s English stopword list to eliminate high-frequency, low-information function words; and (v) WordNet lemmatization to reduce morphological variants to a shared base form. Unlike email preprocessing, no HTML stripping step is required, as SMS messages are plain-text by nature. The output of this stage is a cleaned token sequence for each message.

Stage 3 — TF-IDF Feature Extraction. Cleaned token sequences are vectorized using scikit-learn’s `TfidfVectorizer`, which computes Term Frequency–Inverse Document Frequency (TF-IDF) weights over the full training corpus. Both unigrams and bigrams are included in the feature space to capture individual tokens and two-word phrase-level patterns characteristic of SMS spam (e.g., “free txt”, “call now”, “won prize”, “claim reward”). A maximum feature cap (`max_features`) is

applied to bound vocabulary size given the short-text nature of the corpus, and sublinear TF scaling (`sublinear_tf=True`) is enabled to reduce the disproportionate influence of very frequent terms. The result is a sparse document-term matrix used as input to all downstream classifiers. Given the brevity of SMS messages, the resulting feature vectors are notably sparser than those produced from document-length text, which informs our choice to include bigrams as an enrichment strategy.

Stage 4 — ML Classifier Training. Three supervised classifiers are trained on identical TF-IDF feature representations, enabling a controlled algorithmic comparison: (i) Multinomial Naive Bayes (`MultinomialNB`), a probabilistic baseline that is particularly well-suited to short-text classification due to its low sample complexity and robustness to sparse feature vectors; (ii) Logistic Regression with L2 regularization, a linear discriminative classifier that directly models the posterior probability of the spam class; and (iii) Linear Support Vector Machine (`LinearSVC`), a margin-based classifier effective in high-dimensional sparse feature spaces typical of text data. The dataset is partitioned using a stratified 80/20 train-test split to preserve the natural class imbalance in both subsets. Stratified 5-fold cross-validation is applied during training to reduce variance in model selection and provide a more reliable estimate of generalization performance.

Stage 5 — Model Evaluation. Each classifier is evaluated on the held-out test set using accuracy, precision, recall, and F1-score computed separately for the spam and ham classes. Given the class imbalance in the dataset, macro-averaged and weighted-averaged variants of each metric are also reported to avoid the distortion that majority-class bias can introduce into simple accuracy figures. Confusion matrices

are generated for each model to visualize the distribution of true positives, false positives, true negatives, and false negatives. For this application, precision on the spam class is treated as the primary optimization criterion: a false positive (a legitimate message incorrectly flagged as spam) carries a higher practical cost than a false negative (a missed spam message). The model with the best spam-class F1-score, with tie-breaking on precision, is selected as the final classifier.

Output. The system produces a binary classification label (“spam” or “ham”) for each input message, along with a confidence score derived from the classifier’s posterior probability estimate. In addition to per-message labels, the system generates a diagnostic report summarizing observed spam patterns: the most discriminative TF-IDF terms for the spam class, the distribution of predicted confidence scores across both classes, and a descriptive breakdown of recurring spam message patterns (e.g., prize/reward lures, urgent call-to-action phrases, URL-bearing promotional texts).

7.4 Architecture Developed So Far

At the time of this progress report, the data acquisition and preprocessing stages of the pipeline (Stages 1–2) have been completed and validated. The UCI SMS Spam Collection dataset has been loaded, inspected, and encoded into a unified labeled DataFrame with consistent binary class encoding (spam = 1, ham = 0). Class distribution statistics have been verified: 747 spam messages (13.4%) and 4,825 ham messages (86.6%), confirming the imbalanced nature of the dataset and motivating the use of stratified splitting and weighted evaluation metrics.

The text preprocessing sub-pipeline has been fully implemented and tested on the complete dataset. Lowercasing, punctuation and number removal, tokenization, stopword filtering, and lemmatization all function correctly, producing clean token sequences of the expected form. Spot-checking of preprocessed outputs confirms that known spam vocabulary (e.g., “free”, “win”, “prize”, “claim”, “txt”) is preserved and prominent after processing, while stopwords and punctuation are correctly stripped.

The TF-IDF vectorization layer (Stage 3) has also been implemented using scikit-learn’s `TfidfVectorizer` with unigram and bigram support. Initial exploratory runs confirm that the vectorizer correctly encodes the preprocessed SMS tokens into a sparse document-term matrix, and inspection of the highest-weighted features in the spam class shows strong alignment with expected spam vocabulary: tokens such as “free”, “call”, “txt”, “won”, “prize”, “claim”, and bigrams such as “free txt” and “call now” appear among the top-ranked features, providing early evidence that the feature representation is capturing meaningful signal.

All three classifiers (Stage 4) have been instantiated and are ready for full-scale training. Preliminary runs on a small development subset confirm that all three models compile and produce predictions without error. Full training and formal evaluation on the complete dataset are in progress.

Both the metric comparison as well as the confusion matrices can be found below in Figures 6 and 7.

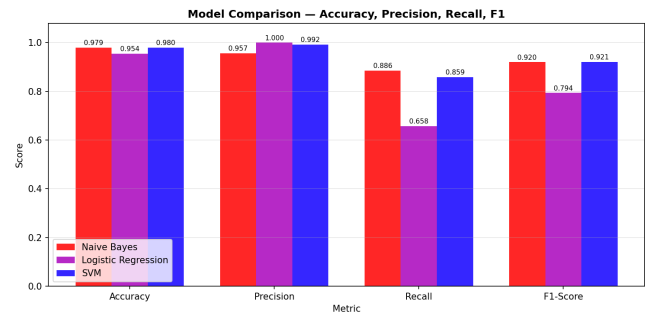


Figure 6: Initial comparison of metrics between different models

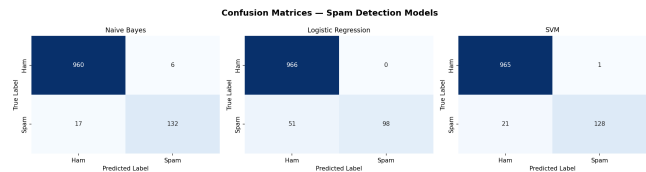


Figure 7: Confusion matrices for each model

The data above confirms some known trends regarding the models. Logistic Regression is best when it comes to precision, which means that it will never mark a legitimate email as spam. However, this results in a tradeoff with F1-score, as it is more susceptible to mark spam emails as ham. Both Naive Bayes and SVM are more well-rounded models.

Conducting full-scale and developing more in-depth metric analysis may give us better insight into which model works best.

7.5 What Remains to Be Developed

Several components of the system remain under active development. First, full-scale classifier training on the complete dataset has not yet been completed. Once full training runs are finished, cross-validation scores and held-out test metrics will be computed for all three classifiers and compared systematically.

Second, the evaluation and reporting modules (Stage 5 and the Output stage) are only partially implemented. Confusion matrix generation and per-class metric computation

are functional, but the final diagnostic spam-pattern report (including top discriminative feature visualization and spam message pattern analysis) is still under development.

Third, hyperparameter tuning has not yet been performed. For Logistic Regression and LinearSVC, the regularization strength (C parameter) will be explored using grid search with cross-validation. For the TF-IDF vectorizer, the effect of varying `max_features`, `n-gram` range, and `sublinear_tf` will also be systematically evaluated to find the optimal feature configuration for the short-text domain.

Finally, the class imbalance handling strategy requires further investigation. Preliminary results suggest that class-weighted training (`class_weight='balanced'` in scikit-learn) may improve recall on the minority spam class at some cost to precision; the trade-off between these metrics will be quantified and documented in the final evaluation.

7.6 Revised Scope and Realistic Outcome

Following the pivot from email to text message spam detection, the project scope has been updated accordingly. The core technical pipeline (preprocessing, TF-IDF vectorization, multi-model training, and evaluation) remains unchanged in structure, but all dataset references, preprocessing design decisions, and domain-specific considerations now reflect the properties of SMS text rather than email. The UCI SMS Spam Collection provides a well-established, publicly available benchmark that is appropriate in size and label quality for the scope of this project.

The three primary deliverables (a fully trained and evaluated spam classifier, a comparative multi-model analysis, and a confusion-matrix-backed evaluation report) remain firmly within scope and on track. The spam pattern diagnostic report is retained as a deliverable but scoped to keyword-based pattern analysis rather than a learned secondary classifier, given the timeline.

Working with the SMS dataset has given us a more grounded view of the challenges ahead. The short-text nature of SMS messages produces notably sparse TF-IDF vectors, and the 13% class imbalance means that naive accuracy is not a reliable proxy for model quality. These factors inform our expectation that macro-averaged F1, rather than accuracy, will be the most meaningful summary metric, and we have adjusted our evaluation framework accordingly. Target spam-class F1 is set at 0.90 or above, consistent with published benchmarks on this dataset using comparable methods.

8 Future Work

Our next steps include fully implementing and comparing our classification models of Naive Bayes, Logistic Regression, and SVM by using accuracy, precision, recall, and F1-score.

In addition, we will continue refining preprocessing, experiment with feature extraction settings, and complete hyperparameter tuning to improve classification quality. We also plan to complete the spam pattern diagnostic report and deepen our analysis of classification errors.

This milestone focuses on building the foundation of our system, including preprocessing and understanding the dataset, to support later modeling and evaluation.

9 GitHub Repository Link

Here is the link to our GitHub repository:

<https://github.com/hannahm99/sp26-cs410-group15-email-spam-detector>